

3DV HW2

Name: 王子轩

Email: wang-zx23@mails.tsinghua.edu.cn

3DV HW2

1 Volume Rendering

1.1 Ray sampling

1.2 Point sampling

1.3 Theory of transmittance calculation

1.5 Train my own

2 Single Image to 3D

2.1 3D losses

2.2 Network design

(a) Implementation

(b) Results and visualization for two loss implementation

Training results for clean dataset

Under clean dataset, HD-trained best model visualization look like this.

Under noisy dataset, CD-loss trained best model visualization look like this

(c) Noisy dataset for training

(d) Why not use HD ?

3 Surface Reconstruction

3.0 Introduction

3.1 Network implementation

3.2 Marching tetrahedra

3.3 Results

Mesh&sampled point cloud results

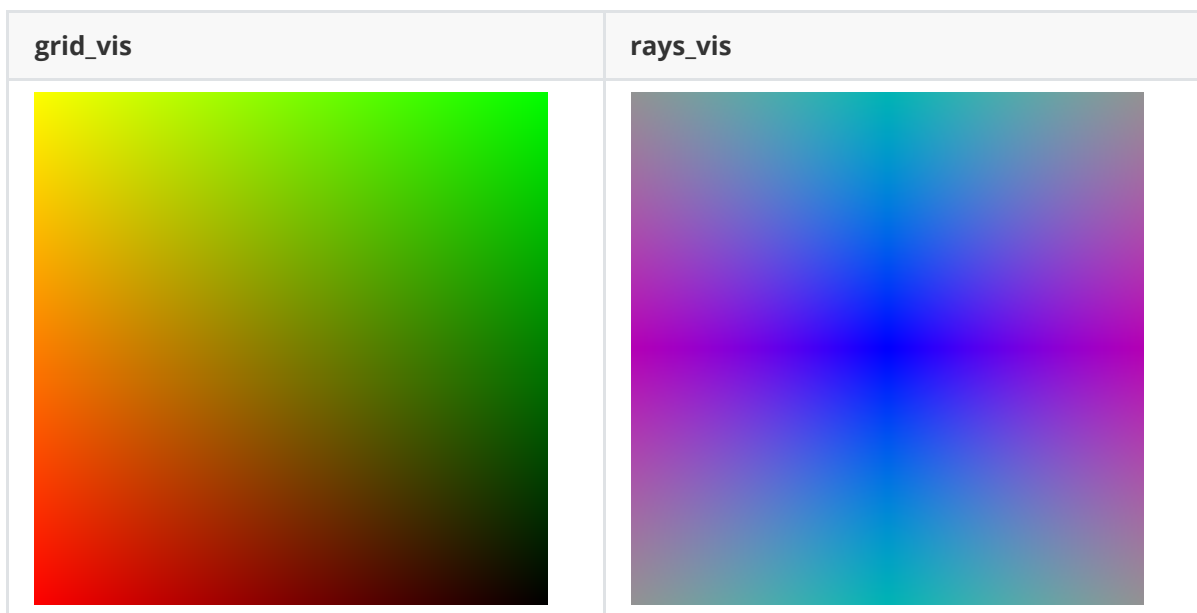
1 Volume Rendering

The whole project file (including code, results...) is in `Problem1`.

- All the code has been completed in the `src` folder, and I trained my model following the guide in the PDF, and the logs are in the `outputs` folder.

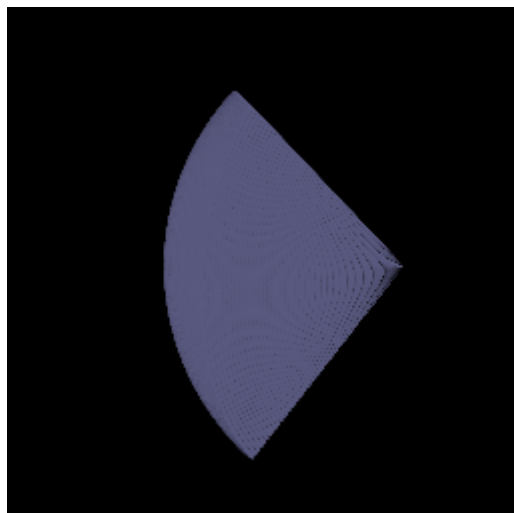
1.1 Ray sampling

`my_images`



1.2 Point sampling

my_images



1.3 Theory of transmittance calculation

2. Sure, let's use LaTeX formatting for the mathematical expressions and keep the explanation concise.

$$\text{Transmittance} = e^{-\int \sigma ds}$$

Given:

- $\sigma_1 = 3$, length = 2m
- $\sigma_2 = 10$, length = 1m
- $\sigma_3 = 1$, length = 4m

Calculate transmittance for each segment:

$$T_1 = e^{-\sigma_1 \cdot \text{length}_1} = e^{-3 \cdot 2}$$

$$T_2 = e^{-\sigma_2 \cdot \text{length}_2} = e^{-10 \cdot 1}$$

$$T_3 = e^{-\sigma_3 \cdot \text{length}_3} = e^{-1 \cdot 4}$$

Total transmittance:

$$T = T_1 \cdot T_2 \cdot T_3 = e^{-6} \cdot e^{-10} \cdot e^{-4}$$

Combine exponents:

$$T = e^{-20}$$

Final answer:

$$e^{-20}$$

images/render_cubes



images/render_nerf



1.5 Train my own

Find my result in `images/render_mynerf`



2 Single Image to 3D

The whole project file (including code, results...) is in `Problem2`.

2.1 3D losses

We use $d(p_i, q_j) = \|p_i - q_j\|_2$ to represent the Euclidean distance between p_i and q_j .

- Hausdorff Distance between P and Q is defined as
$$h(P, Q) = \max\{d(P, Q), d(Q, P)\},$$

- Chamfer Distance between P and Q is defined as $c(P, Q) = d(P, Q) + d(Q, P)$.
 where $d(P, Q) = \max_{p_i \in P} [\min_{q_j \in Q} d(p_i, q_j)]$ and $d(Q, P) = \max_{q_j \in Q} [\min_{p_i \in P} d(q_j, p_i)]$.

```
# loss.py
def chamfer_loss(point_set_a, point_set_b):
    distance_matrix = pairwise_distances(point_set_a, point_set_b)
    dist_a_to_b = torch.mean(torch.sqrt(distance_matrix.min(1)[0]))
    dist_b_to_a = torch.mean(torch.sqrt(distance_matrix.min(2)[0]))
    return dist_a_to_b + dist_b_to_a

def hausdorff_loss(point_set_a, point_set_b):

    distance_matrix = pairwise_distances(point_set_a, point_set_b)
    max_min_a_to_b = torch.max(torch.min(distance_matrix, dim=2)[0])
    max_min_b_to_a = torch.max(torch.min(distance_matrix, dim=1)[0])
    return torch.max(max_min_a_to_b, max_min_b_to_a)
```

2.2 Network design

(a) Implementation

The PointCloudAutoEncoder model is designed as a convolutional autoencoder for generating 3D point clouds from 2D images. The architecture consists of three main components:

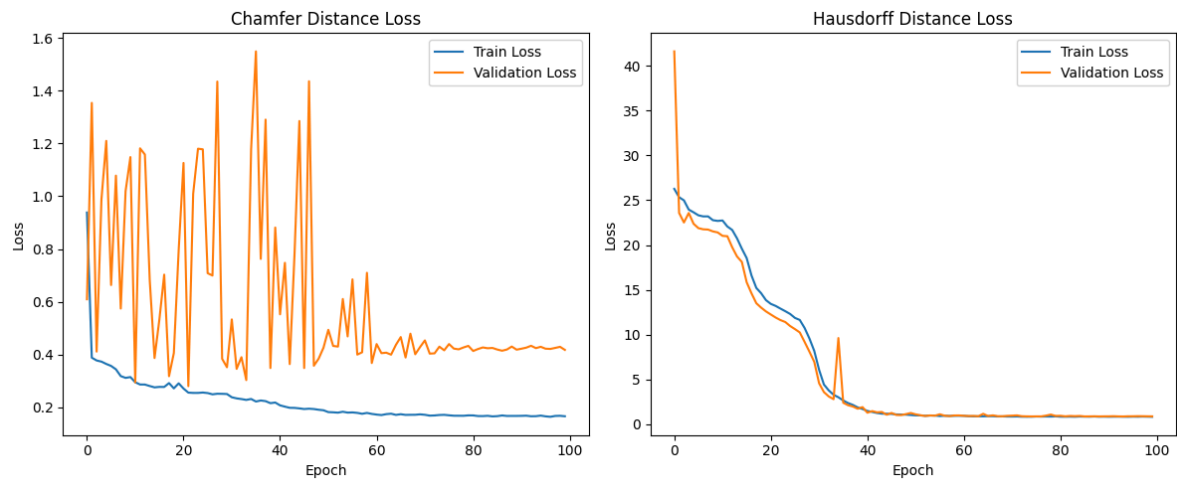
1. **Encoder:** A deep convolutional network that processes input images (batch_size, 3, H, W) through a series of residual blocks. Each block contains two convolutional layers with batch normalization and ReLU activation, followed by downsampling. The encoder ends with global average pooling to produce a 512-dimensional feature vector.
2. **Fully Connected Layer:** A linear layer that maps the 512D encoder output to a user-specified feature dimension (default 512).
3. **Decoder:** A fully connected network that reconstructs the point cloud from the latent feature vector. It progressively upsamples the features through three linear layers with ReLU activations, finally outputting 3D coordinates for a fixed number of points (default 1024).

The model processes input images through the encoder, transforms the features through the FC layer, and then reconstructs the point cloud through the decoder. The final output is reshaped to (batch_size, num_points, 3) format, representing the predicted 3D point coordinates.

This architecture enables the model to learn meaningful representations of 3D shapes from 2D images and generate corresponding point cloud reconstructions. You can check the `src/model.py` for implementation details.

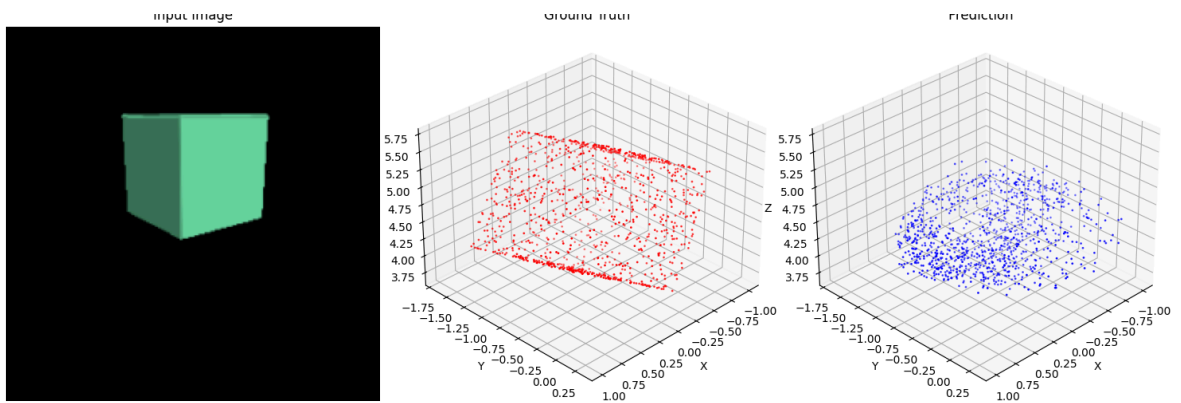
(b) Results and visualization for two loss implementation

Training results for clean dataset

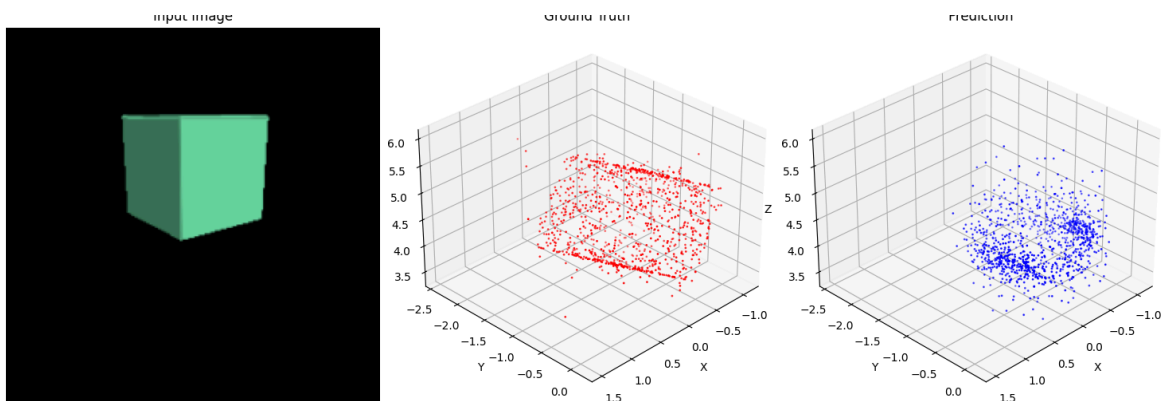
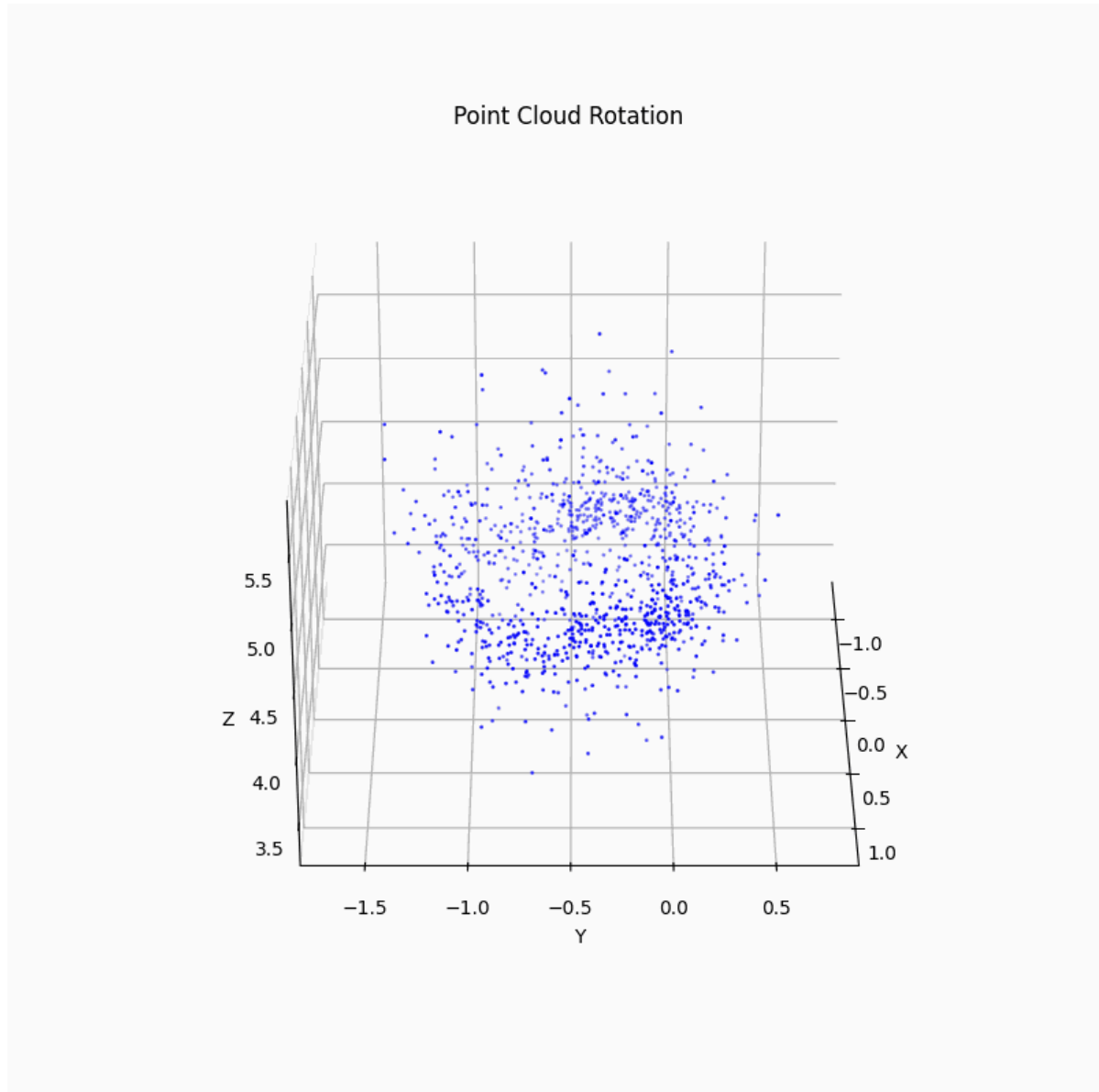


Under clean dataset, HD-trained best model visualization look like this.

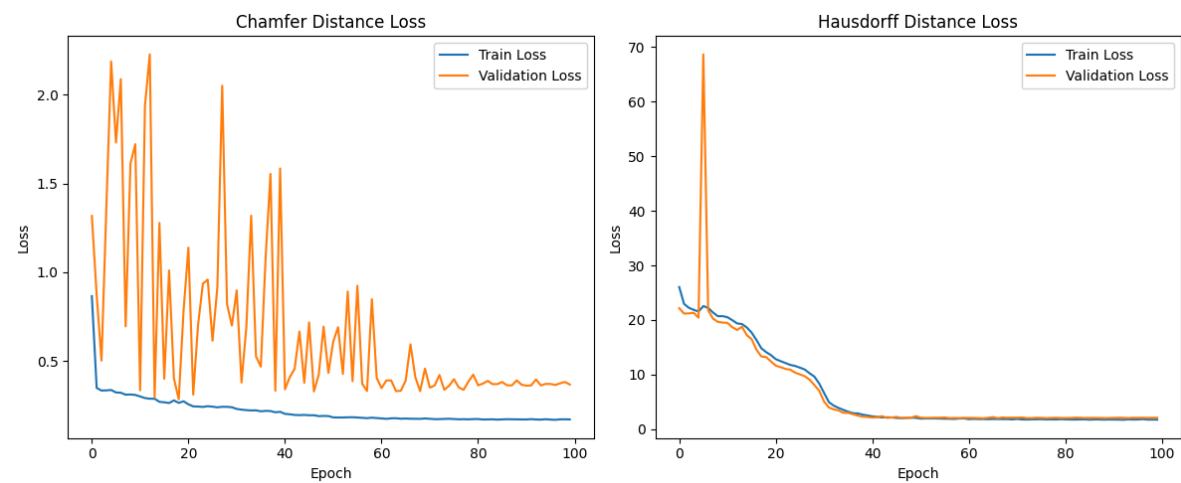




Under noisy dataset, CD-loss trained best model visualization look like this



(c) Noisy dataset for training



SCORE	CD	HD
CLEAN	0.346256	0.465091
NOISY	0.337933	1.048274

(d) Why not use HD ?

HD loss measures the maximum minimum distance between two point clouds, rendering it highly sensitive to outliers. In scenarios where reconstructed point clouds exhibit sparse regions, noise, or uneven density—common artifacts in single-view reconstruction—a single outlier can disproportionately dominate the loss value, leading to unstable optimization and suboptimal geometric fidelity. The gradient dynamics of HD loss are inherently unstable. By focusing solely on the worst-case point pair, HD loss produces sparse gradients that fail to guide the network toward globally coherent shape recovery, often resulting in erratic parameter updates or local minima. In contrast, CD loss distributes gradients across all points, enabling smoother convergence by balancing local and global geometric alignment. HD loss prioritizes extreme local discrepancies over holistic shape consistency—a misalignment with the primary objective of single-image reconstruction, which emphasizes overall structural accuracy rather than penalizing isolated errors. For instance, even if 99% of a reconstructed point cloud aligns perfectly with the ground truth, HD loss would still assign a high penalty due to a single distant outlier, discouraging the network from preserving valid global structures.

3 Surface Reconstruction

The whole project file (including code, results...) is in `Problem3`.

3.0 Introduction

Project Structure

```
├─ assets
|   └─ image.png
├─ configs
|   └─ config.yaml
├─ data
|   └─ pcs
```

```

|   ├── pcs_normalized # The original pcs file are out of [-0.5, 0.5] range.
|   └── tets
|       ├── 100_compress.npz
|       └── ...
├── outputs # These are the results
├── README.md
├── requirements.txt
├── results
|   ├── pc_ranges_report.txt
|   └── tet_ranges_report.txt
├── scripts          # some helper scripts
|   ├── analyze_pc_ranges.py
|   ├── analyze_tet_ranges.py
|   ├── normalize_pc.py
|   └── run_grid_search.sh
├── src
|   ├── data.py
|   ├── model.py
|   ├── test_modules.py # I used it to examine the correctness of the pipeline's
componen
|   ├── train.py
|   └── utils.py

```

Training pipeline

The training pipeline employs a neural network architecture to predict both Signed Distance Field (SDF) values and deformation vectors for 3D tetrahedral meshes. The network takes vertex coordinates as input and outputs SDF values that define the implicit surface and deformation vectors that adjust vertex positions. The Marching Tetrahedra (MT) algorithm is then utilized to convert these SDF values into explicit triangle meshes. To optimize the network, point samples are uniformly drawn from the generated mesh surface and compared against the target point cloud using Chamfer Distance as the primary loss metric. Additionally, a Laplacian regularization term is incorporated to ensure smoothness in the predicted deformations. The combined loss function drives the optimization process, enabling the network to learn accurate SDF and deformation predictions that closely match the target geometry. This pipeline effectively bridges the gap between implicit surface representations and explicit mesh generation, while maintaining differentiable optimization throughout the process. I use hydra to track the results of different combination of paramameter, you can refer to `configs/config.yaml`

3.1 Network implementation

The model implementation consists of three main components: PositionalEncoding, MLPNetwork, and Conv3DNetwork, integrated into a unified DMTetModel. The PositionalEncoding implements Fourier feature mapping to transform 3D coordinates into higher-dimensional space, and supports configurable frequency bands and optional inclusion of raw coordinates. In this experiment I found that features skip connections and leaky ReLU activations are very important. Initially I used ReLu, but the SDF output due to the output range, thus can not poduce any predicted mesh sufaces and can not be optimized. 3D convolutional network operating on volumetric grid, and I Implements downsampling and upsampling convolution blocks. **You can check `src/model.py` for this part's implementation.** A specific finding is that the frequency used to encode the position of the point cloud shoudn't be high. As the following, I found that set `num_freqs` to 1 is better than any implemmentions.

```

class PositionalEncoding(nn.Module):
    def __init__(self, num_freqs=10, include_input=True):
        super(PositionalEncoding, self).__init__()
        self.num_freqs = num_freqs
        self.include_input = include_input
        self.funcs = [torch.sin, torch.cos]
        self.out_dim = 0
        if self.include_input:
            self.out_dim += 3
        self.out_dim += 2 * 3 * self.num_freqs

    def forward(self, x):
        orig_shape = list(x.shape)
        x = x.reshape(-1, 3)
        encoded = []
        if self.include_input:
            encoded.append(x)
        for freq_idx in range(self.num_freqs):
            freq = 2.0 ** freq_idx
            for func in self.funcs:
                encoded.append(func(x * freq * np.pi))
        encoded = torch.cat(encoded, dim=-1)
        encoded = encoded.reshape(orig_shape[:-1] + [self.out_dim])
        return encoded

```

The higher of the value of `num_freqs`, the more difficult of the optimization process, which leads to poor smoothness. However it leads model focus on small detail of the surface like the apple. This below is the results observed. By the way, due to limited computational resources, only 10,000 points are sampled per epoch during training. In the image below, a strange hourglass shape appears at the base of the apple during low-frequency encoding. However, I am not entirely sure about the exact cause. I suspect that this might be due to the fact that fewer points are sampled during training compared to the target points, which causes difficulties in optimization within the CD loss. Increasing the number of sampled points may help resolve this issue.



About the memory usage, Conv3DNetwork requires significant memory for large grid sizes. MLPNetwork is more memory-efficient for sparse inputs

3.2 Marching tetrahedra

This part is the core algorithm of the whole pipeline. **You can check `utils.py` for implementation detail.** MT converts Signed Distance Field (SDF) values into triangle meshes. The implementation is encapsulated within the `MarchingTetrahedra` class, which inherits from `torch.nn.Module` to ensure compatibility with PyTorch's automatic differentiation system. The algorithm begins by initializing lookup tables and variables essential for the MT process. These

include a triangle table that defines the triangulation patterns for all possible configurations of tetrahedron vertices, a table indicating the number of triangles generated for each configuration, and base tetrahedron edges used for edge indexing. The `v_id` variable is used to encode the occupancy state of tetrahedron vertices. The core of the MT algorithm is implemented in the `forward` method. It processes input vertices, tetrahedron indices, and SDF values to generate mesh vertices and faces. The algorithm first determines the occupancy state of each tetrahedron vertex based on the SDF values. Valid tetrahedra are identified as those with at least one occupied and one unoccupied vertex. For each valid tetrahedron, the algorithm identifies unique edges that intersect the surface and computes the intersection points using linear interpolation. These intersection points become the vertices of the generated mesh. The triangulation process uses the precomputed lookup tables to determine the appropriate triangle configurations based on the occupancy pattern of each tetrahedron. The algorithm handles both single and double triangle configurations, ensuring correct mesh generation for all cases. The resulting mesh vertices and faces are returned as lists of tensors, maintaining batch compatibility. This implementation provides a differentiable pipeline for converting SDF representations to explicit triangle meshes, enabling gradient-based optimization of 3D shapes in neural network applications. The use of PyTorch ensures efficient computation on both CPU and GPU devices, making it suitable for large-scale 3D reconstruction tasks.

```
def forward(self, vertices, tets, sdf):
    """
    Execute Marching Tetrahedra algorithm
    Args:
        vertices: Vertex coordinates [batch_size, num_vertices, 3]
        tets: Tetrahedron indices [batch_size, num_tets, 4]
        sdf: SDF values per vertex [batch_size, num_vertices, 1]
    Returns:
        mesh_vertices: Mesh vertices [batch_size, num_mesh_vertices, 3]
        mesh_faces: Mesh faces [batch_size, num_mesh_faces, 3]
    """
    batch_size = vertices.shape[0]
    device = vertices.device

    triangle_table = self.triangle_table.to(device)
    num_triangles_table = self.num_triangles_table.to(device)
    base_tet_edges = self.base_tet_edges.to(device)
    v_id = self.v_id.to(device)

    mesh_vertices = []
    mesh_faces = []

    for b in range(batch_size):
        pos_nx3 = vertices[b]
        sdf_n = sdf[b].squeeze(-1)
        tet_fx4 = tets[b]

        with torch.no_grad():
            occ_n = sdf_n > 0
            occ_fx4 = occ_n[tet_fx4.reshape(-1)].reshape(-1, 4)
            occ_sum = torch.sum(occ_fx4, -1)
            valid_tets = (occ_sum > 0) & (occ_sum < 4)

            if not valid_tets.any():
```

```

        mesh_vertices.append(torch.zeros((0, 3), device=device))
        mesh_faces.append(torch.zeros((0, 3), dtype=torch.long,
device=device))
        continue

    all_edges = tet_fx4[valid_tets][:,
base_tet_edges.reshape(-1)].reshape(-1, 2)
    all_edges = self._sort_edges(all_edges)
    unique_edges, idx_map = torch.unique(all_edges, dim=0,
return_inverse=True)

    unique_edges = unique_edges.long()
    mask_edges = occ_n[unique_edges.reshape(-1)].reshape(-1, 2).sum(-1)
    == 1
    mapping = torch.ones((unique_edges.shape[0]), dtype=torch.long,
device=device) * -1
    mapping[mask_edges] = torch.arange(mask_edges.sum(),
dtype=torch.long, device=device)
    idx_map = mapping[idx_map]

    interp_v = unique_edges[mask_edges]

    edges_to_interp = pos_nx3[interp_v.reshape(-1)].reshape(-1, 2, 3)
    edges_to_interp_sdf = sdf_n[interp_v.reshape(-1)].reshape(-1, 2, 1)
    edges_to_interp_sdf[:, -1] *= -1

    denominator = edges_to_interp_sdf.sum(1, keepdim=True)
    edges_to_interp_sdf = torch.flip(edges_to_interp_sdf, [1]) / denominator
    verts = (edges_to_interp * edges_to_interp_sdf).sum(1)

    idx_map = idx_map.reshape(-1, 6)

    tetindex = (occ_fx4[valid_tets] * v_id.unsqueeze(0)).sum(-1)
    num_triangles = num_triangles_table[tetindex]

    faces_list = []

    if (num_triangles == 1).any():
        faces_1 = torch.gather(
            input=idx_map[num_triangles == 1], dim=1,
            index=triangle_table[tetindex[num_triangles == 1]][:, :3]
        ).reshape(-1, 3)
        faces_list.append(faces_1)

    if (num_triangles == 2).any():
        faces_2 = torch.gather(
            input=idx_map[num_triangles == 2], dim=1,
            index=triangle_table[tetindex[num_triangles == 2]][:, :6]
        ).reshape(-1, 3)
        faces_list.append(faces_2)

    if faces_list:
        faces = torch.cat(faces_list, dim=0)
        mesh_vertices.append(verts)
        mesh_faces.append(faces)
    else:

```

```

mesh_vertices.append(torch.zeros((0, 3), device=device))
mesh_faces.append(torch.zeros((0, 3), dtype=torch.long,
device=device))

return mesh_vertices, mesh_faces

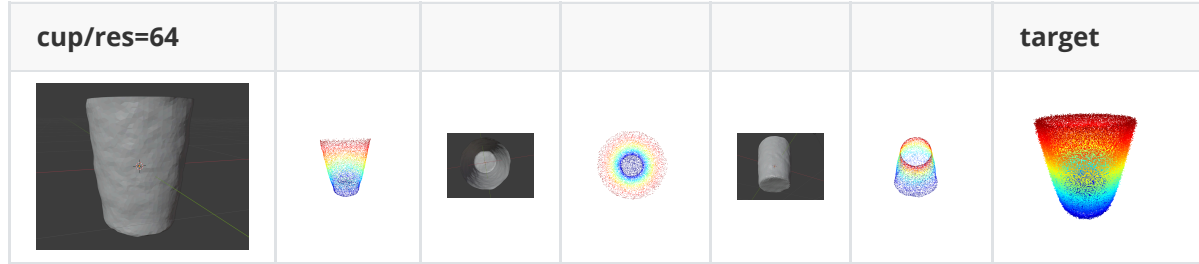
```

3.3 Results

Mesh&sampld point cloud results

Below results are rendered by Blender 4.3.

10000 (< number of target input)points are sampled during the pipeline



Below results are rendered by open3d lib.

The loss is the add of 2 components $\mathcal{L} = \mathcal{L}_{CD} + \lambda_{reg}\mathcal{L}_{reg}$

Since both loss ration λ_{reg} and hyper parameter **{grid_res}** are required to be here in the format.

λ_{reg}	64	70	80	90	100	128
0.1						
0.5						
1						
2						N/A

The size of the Laplacian regularization term (λ_{reg}) influence the trade-off between smoothness and accuracy. When λ_{reg} is small (close to 0), the Chamfer Distance dominates the optimization, resulting in meshes that closely match the original point cloud's geometric details, preserving local features. However, this can lead to noise and unsmooth surfaces, especially in noisy point clouds, causing potential overfitting. A moderate λ_{reg} strikes a balance, preserving key structures while suppressing noise, yielding smooth and reasonably detailed reconstructions. In this case, the Laplacian regularization helps to constrain vertex distribution, preventing local distortions. Conversely, an excessively large λ_{reg} introduces strong smoothing constraints that create overly uniform surfaces, reducing the impact of noise. While this helps mitigate noise, excessive smoothing can cause the loss of important geometric features, such as sharp edges and depressions, resulting in a "blurred" effect and reduced reconstruction accuracy. For clean point clouds, a smaller λ_{reg} is preferred to preserve details, while a larger λ_{reg} (e.g., 0.5-1.0) enhances

robustness in noisy point clouds. If reconstruction shows shrinkage or loss of details, λ_{reg} should be reduced. Ultimately, the selection of λ_{reg} should be guided by mesh visualization and quantitative metrics, such as Chamfer Distance (CD) values and curvature changes, to achieve an optimal balance between fidelity and smoothness. **For example, in the case of 1-128, reconstruction failures** indicate that an overly large λ_{reg} dominated the optimization, preventing the reconstructed mesh from fitting the original point cloud data effectively. This can be explained by the fact that when λ_{reg} is too large, the gradient of the Laplacian term dominates the optimization, diminishing the Chamfer Distance's ability to capture geometric details. While Laplacian regularization aims to smooth the mesh surface and reduce irregular vertex distributions, overly strong smoothing suppresses the mesh's ability to capture local features, particularly at high resolutions (e.g., gridres=128), where this effect is more pronounced.

Success Case (0.5-100)	Failure Case(1-128)	Mid process case: PC after 350 epoch
		

You can check the `outputs` folder to access the full results.